Conversational SHL Assessment Recommender - Approach Summary

Candidate: SHL Assignment Submission

1) Design Choices

Goal: build a recruiter-facing API that asks clarifying questions, returns grounded SHL recommendations, refines results as constraints change, and compares assessments with catalog evidence.
Architecture: FastAPI backend + React frontend, stateless chat API (full message history each request).
Retrieval-first design: catalog decides recommendations; LLM is optional and used only for response rewriting.
Explicit conversation states: clarifying, recommending, refining, comparing, refuse.
Structured contract: reply, recommendations, end_of_conversation, plus state/source/model metadata and confidence.

2) Retrieval Setup

Data source: normalized local SHL catalog JSON.
User query parsing extracts role, seniority, test types, and skills.
Scoring uses weighted overlap: role (5), seniority (4), test type (6), skills/keywords (4), plus direct type boost.
Results are ranked and returned as top matches, each with normalized confidence (score / max_score).
Comparison flow uses catalog name matching against mentioned assessments.

3) Prompt Design

LLM: Groq llama-3.1-8b-instant (optional).
Prompt includes intent, user request, base deterministic answer, and grounded match list.
System guardrail: never invent assessments, URLs, or unsupported claims.
Fallback behavior: if LLM unavailable/fails, return deterministic catalog response.

Evaluation, Failures, and Improvements

4) Evaluation Method
Automated tests validate API schema and conversational behavior: clarify, recommend, refine, compare, and refuse.
Groq fallback and schema stability are tested to ensure model use does not break contract.
Evaluation harness runs both custom traces and recruiter persona suite.
Persona suite covers: Java mid-level cognitive request, refinement with personality addition, data analyst screening, named comparison, vague intake clarification, and out-of-scope refusal.
Captured metrics: recommendation_count, state, reply_source, llm_model, average_confidence.

5) What Did Not Work Initially
Unconstrained generation for recommendations increased hallucination risk.
Overly strict reply-text assertions became brittle once Groq paraphrasing was enabled.
Frontend metadata badges added noise for demo/interview flow and were removed based on feedback.

6) How Improvement Was Measured
Test pass rate after each change (final backend status: all tests passing).
Persona-trace outcomes to verify correct state transitions and recommendation relevance.
Groundedness checks: recommendations and URLs must come from catalog data.
Operational visibility via response metadata and confidence values.

Summary: The final system balances deterministic retrieval reliability with optional LLM readability improvements.